1.1.25

Vishwambhar - EE25BTECH11025

26th March, 2026

Question

The state-space representation of a separately excited DC servo motor dynamics is given as:

$$\frac{d}{dt}\begin{pmatrix}\omega \\ i_a\end{pmatrix} = \begin{pmatrix}-1 & 1 \\ -1 & -10\end{pmatrix}\begin{pmatrix}\omega \\ i_a\end{pmatrix} + \begin{pmatrix}0 \\ 10\end{pmatrix} u \tag{1}$$

where ω is the speed, i_a is the armature current, and u is the armature voltage.
The transfer function $\frac{\omega(s)}{u(s)}$

1. $\frac{10}{s^2+11s+11}$
2. $\frac{1}{s^2+11s+11}$
3. $\frac{10s+10}{s^2+11s+11}$
4. $\frac{1}{s^2+s+1}$

Given

$$\dot{\mathbf{x}} = A\mathbf{x} + \mathbf{b}u \tag{2}$$

where,

$$\mathbf{x} = \begin{pmatrix} \omega \\ i_a \end{pmatrix} \tag{3}$$

$$A = \begin{pmatrix} -1 & 1 \\ -1 & -10 \end{pmatrix} \tag{4}$$

$$\mathbf{b} = \begin{pmatrix} 0 \\ 10 \end{pmatrix} \tag{5}$$

Taking Laplace Transform

$$sX(s) = AX(s) + \mathbf{b}U(s) \tag{6}$$
$$(sI - A)X(S) = \mathbf{b}U(s) \tag{7}$$
$$X(s) = (sI - A)^{-1} + \mathbf{b}U(s) \tag{8}$$

we only want ω, so:

$$Y(s) = \mathbf{c}\top X(s) \tag{9}$$
$$Y(s) = \mathbf{c}\top (sI - A)^{-1} bU(s) \tag{10}$$

where,

$$c = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \tag{11}$$

Taking Laplace Transform

$$sX(s) = AX(s) + \mathbf{b}U(s) \tag{12}$$
$$(sI - A)X(S) = \mathbf{b}U(s) \tag{13}$$
$$X(s) = (sI - A)^{-1} + \mathbf{b}U(s) \tag{14}$$

we only want ω, so:

$$Y(s) = \mathbf{c}\top X(s) \tag{15}$$
$$Y(s) = \mathbf{c}\top (sI - A)^{-1} bU(s) \tag{16}$$

where,

$$c = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \tag{17}$$

Computing

Final formula,

$$G(s) = \frac{Y(s)}{U(s)} = \mathbf{c}\top (sI - A)^{-1} b \tag{18}$$

Multiplying Everything,

$$(sI - A)^{-1} b = \frac{1}{s^2 + 11s + 11} \begin{pmatrix} s + 10 & 1 \\ -1 & s + 1 \end{pmatrix} \begin{pmatrix} 0 \\ 10 \end{pmatrix} \tag{19}$$

$$= \frac{1}{s^2 + 11s + 11} \begin{pmatrix} 10 \\ 10(s + 1) \end{pmatrix} G(s) = \frac{10}{s^2 + 11s + 11} \tag{20}$$

$$\tag{21}$$

Converting into Time Domain

Starting from the transfer function of the DC servo motor,

$$G(s) = \frac{\omega(s)}{u(s)} = \frac{10}{s^2 + 11s + 11} \tag{22}$$

$$(s^2 + 11s + 11)\omega(s) = 10u(s) \tag{23}$$

$$s\omega(s) \leftrightarrow \frac{d\omega(t)}{dt} \tag{24}$$

$$s^2\omega(s) \leftrightarrow \frac{d^2\omega(t)}{dt^2} \tag{25}$$

$$\frac{d^2\omega(t)}{dt^2} + 11\frac{d\omega(t)}{dt} + 11\omega(t) = 10u(t) \tag{26}$$

Python Code(Transfer Function)

```
import numpy as np
import control as ctrl

# Define matrices
A = np.array([[-1, 1],
              [-1, -10]])
B = np.array([[0],
              [10]])
C = np.array([[1, 0]])
D = np.array([[0]])

sys_ss = ctrl.StateSpace(A, B, C, D)
sys_tf = ctrl.ss2tf(sys_ss)

print(Transfer Function:)
print(sys_tf)
```

Transfer Function

```
~/Scientific_Programming/1.1.25/codes 2m 29s
venv ❯ python3 solving.py
Transfer Function:
<TransferFunction>: sys[1]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

        10
  ---------------
  s^2 + 11 s + 11
```

Comparing Final Output

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# System definition from Laplace transform
# G(s) = 10 / (s^2 + 11s + 11)
num = [10]
den = [1, 11, 11]

system = signal.TransferFunction(num, den)

# Input sine wave
f = 1 # Hz
omega = 2 * np.pi * f # angular frequency

dt = 0.001
t = np.arange(0, 10, dt)

u = np.sin(omega * t)
```

Comparing Final Output

```
y_rk4 = np.zeros(len(t))
ydot = np.zeros(len(t))
def f1(y, ydot, time):
    return ydot
def f2(y, ydot, time):
    inp = np.sin(omega * time)
    return 10 * inp - 11 * ydot - 11 * y
for i in range(len(t) - 1):
    k1_y = dt * f1(y_rk4[i], ydot[i], t[i])
    k1_v = dt * f2(y_rk4[i], ydot[i], t[i])
    k2_y = dt * f1(y_rk4[i] + k1_y/2,
                   ydot[i] + k1_v/2,
                   t[i] + dt/2)
    k2_v = dt * f2(y_rk4[i] + k1_y/2,
                   ydot[i] + k1_v/2,
                   t[i] + dt/2)
    k3_y = dt * f1(y_rk4[i] + k2_y/2,
                   ydot[i] + k2_v/2,
                   t[i] + dt/2)
```

Comparing Final Output

```
# Laplace output equation
jw = 1j * omega
Gjw = 10 / (jw**2 + 11*jw + 11)

A_L = abs(Gjw)
phi_L = np.angle(Gjw)

print(Laplace Transform Output Equation:)
print(fy_L(t) = {A_L:.4f} sin({omega:.4f} t {phi_L:+.4f}))

# RK4 approximate equation
y_ss = y_rk4[-2000:]
t_ss = t[-2000:]

A_R = (np.max(y_ss) - np.min(y_ss)) / 2

# phase estimation using peak location
peak_index = np.argmax(y_ss)
```

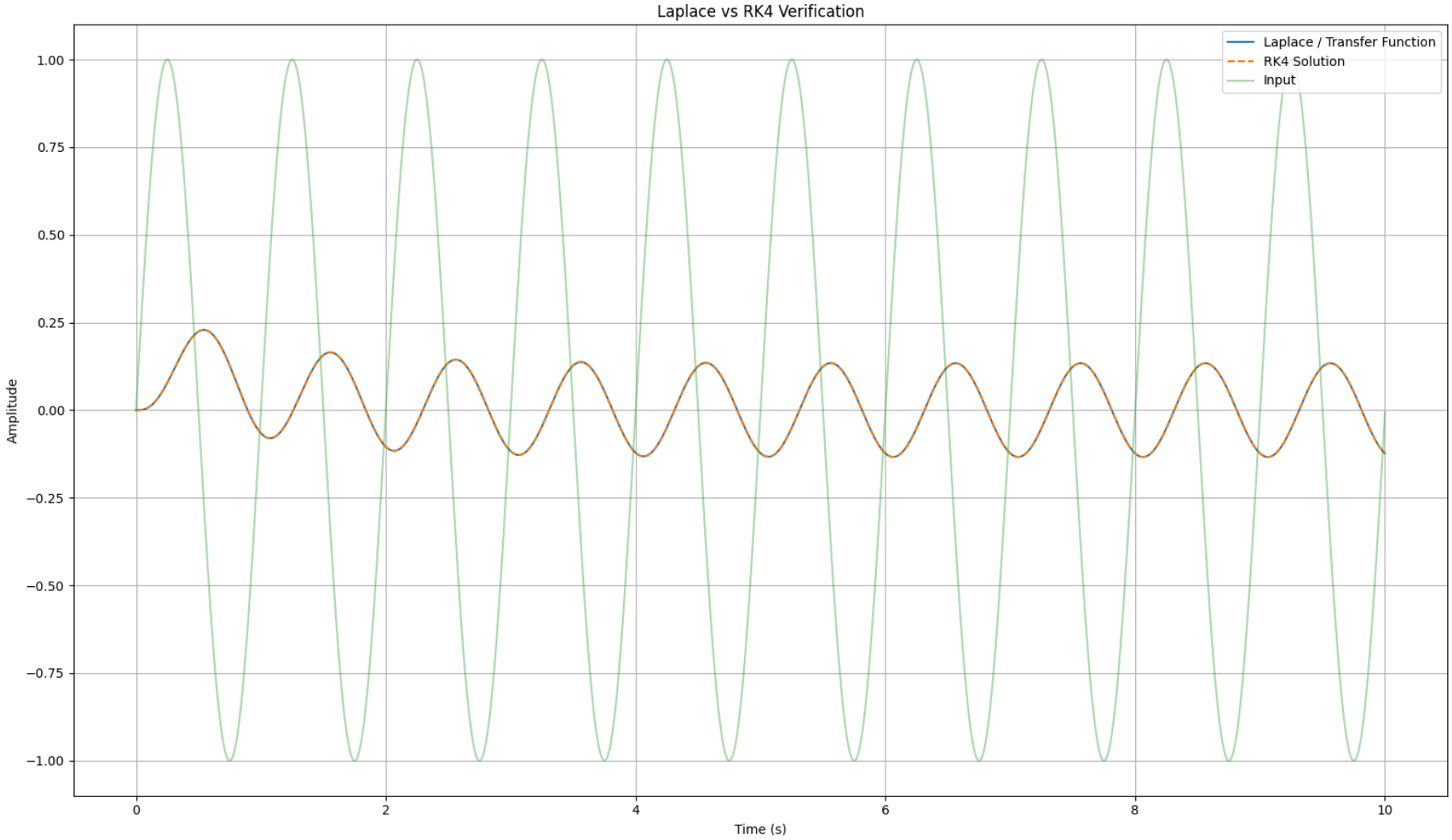
Comparing Final Output

```
phi_R = np.pi/2 - omega * t_peak

print(\nRK4 Approximate Output Equation:)
print(fy_RK4(t) = {A_R:.4f} sin({omega:.4f} t {phi_R:+.4f}))

# Plot comparison
plt.figure(figsize=(10,5))
plt.plot(t_tf, y_tf, label='Laplace / Transfer Function')
plt.plot(t, y_rk4, '--', label='RK4 Solution')
plt.plot(t, u, alpha=0.4, label='Input')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Laplace vs RK4 Verification')
plt.grid(True)
plt.legend()
plt.show()
```

Output Comparison



Output Comparison

```
~/Scientific_Programming/1.1.25/codes
venv ❯ python3 plot.py
Laplace Transform Output Equation:
y_L(t) = 0.1338 sin(6.2832 t -1.9616)

RK4 Approximate Output Equation:
y_RK4(t) = 0.1338 sin(6.2832 t -52.2258)
```